

Session 2.5

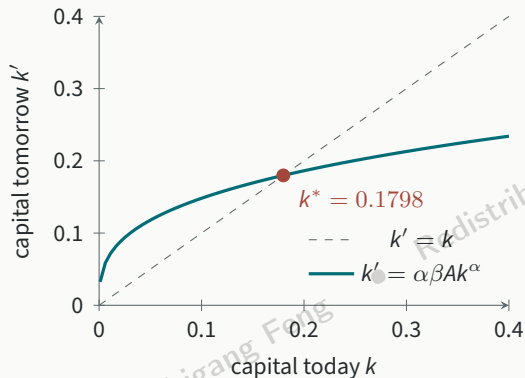
Solving the Growth Model as Supervised Learning on the Euler Equation

The network is the policy, sampled states are the inputs, and the Euler equation supplies the labels



Zhigang Feng

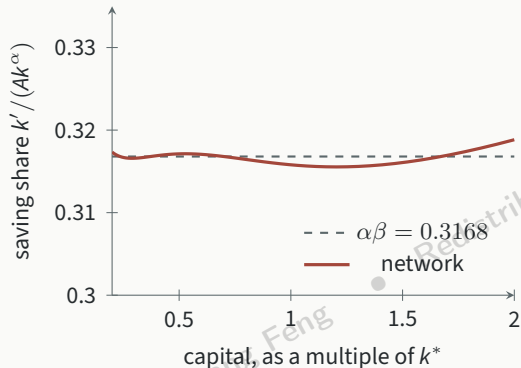
Session 1 Solved This in One Second. Why?



- ▶ The growth model of Session 1.4: choose k' to maximise $\sum_t \beta^t u(c_t)$ subject to $c + k' = Ak^\alpha + (1 - \delta)k$.
- ▶ With log utility and $\delta = 1$ the **policy function** is known: $k' = \alpha\beta Ak^\alpha$.
- ▶ In the Session 1 lab a network recovered it to within 0.44% in under a second.
- ▶ **Fast and accurate. Before trusting the method, ask what it actually had to learn.**

Source: Calibration $\alpha = 0.33, \beta = 0.96, A = 1, \delta = 1$, log utility. Session 1 lab notebook; Brock & Mirman, *JET* 4(3), 1972.

The Reveal: the Target Was One Number



- ▶ Write the policy as a share of output: $k'/(Ak^\alpha) = \alpha\beta$, the same at every k .
- ▶ The network's learned share lies between 0.3155 and 0.3188 everywhere. It learned a constant.
- ▶ One bias term could pass this test. It says the code is right; it says little about approximation.
- ▶ **So this deck climbs a ladder: the same code, on problems where the share is a real function.**

Source: Session 2 lab notebook, block D (rung 0: log utility, $\delta = 1$, $\alpha = 0.33$, $\beta = 0.96$, $A = 1$).

Session 2.5

The Residual as a Loss

Where the labels come from when there are no data



Solving a Model as Supervised Learning: the Dictionary

Supervised learning	Digits (Decks 2.1–2.3)	Growth model (this deck)
inputs x_i	images	sampled states k_i (and z_i)
model f_θ	classifier	the policy $k' = \pi_\theta(k, z)$
prediction error	$\hat{p}$ against the label	the Euler residual at (k_i, z_i)
label	the true digit	zero , at every state

- ▶ Nobody hands us the right k' at each k . The model's own optimality condition says what a right answer must satisfy.
- ▶ Inputs are free: we draw as many states as we like, fresh at every step.
- ▶ **The Euler equation supplies the labels. Everything in Decks 2.3 and 2.4 then applies unchanged.**

The Euler Equation, and Its Unit-Free Residual

- ▶ The **Euler equation**: at an optimum, giving up one unit of consumption today and eating the proceeds tomorrow must leave utility unchanged,

$$u'(c) = \beta E[u'(c') R'], \quad R' = \alpha z' A k'^{\alpha-1} + 1 - \delta.$$

- ▶ Under a candidate policy π_θ both sides can be computed at any state. Their gap is the **residual**.
- ▶ A gap in marginal-utility units means nothing by itself. Make it **unit-free**: divide, and take logs,

$$r_\theta(k, z) = \log \frac{\beta E[u'(c') R']}{u'(c)}.$$

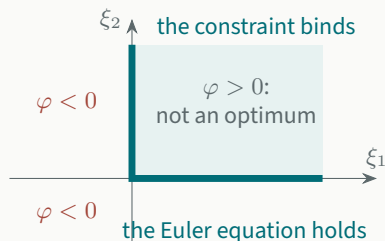
- ▶ $r = 0.01$ reads as a one per cent wedge between the two sides, at any calibration.
- ▶ **The lab code uses exactly this log residual, on all three rungs.**

Necessary, Not Sufficient

- ▶ The Euler equation is a *first-order* condition. The true policy satisfies it. So do other paths.
- ▶ The full solution also needs the **transversality condition**: capital must not be accumulated, or run down, without bound. The Euler equation alone does not impose it.
- ▶ A residual is also *local*: it compares today with tomorrow at one state. Small errors at every state can still compound along a simulated path.
- ▶ A loss built on the Euler equation therefore cannot certify the policy that minimises it.
- ▶ **A small residual is evidence. It is not a proof. Rung 1, later in this deck, measures the difference.**

Source: N. Stokey, R. Lucas, E. Prescott, *Recursive Methods in Economic Dynamics*, 1989, §4.5.

When the Euler Equation Holds as an Inequality

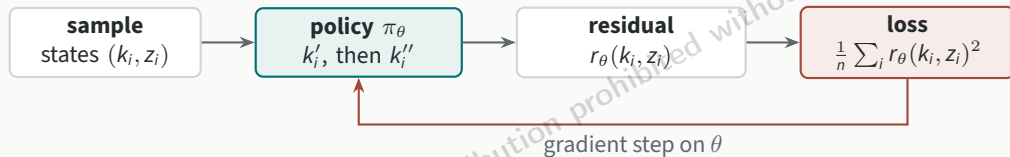


The thick corner is $\varphi = 0$: all three conditions at once. Architecture is exact where the shape can be written in, as the sigmoid saving share is; this residual is general; a penalty is approximate.

Source: A. Fischer, “A special Newton-type optimization method”, *Optimization* 24, 1992; Maliar, Maliar & Winant, *JME* 122, 2021.

- ▶ With $k' \geq \underline{k}$, optimality is a **Kuhn–Tucker** (KKT) system: slack $\xi_1 = k' - \underline{k} \geq 0$, Euler wedge $\xi_2 \geq 0$, $\xi_1 \xi_2 = 0$ — **complementary slackness**, an **occasionally binding** constraint.
- ▶ A squared residual of an equality cannot express an inequality. $\min(\xi_1, \xi_2) = 0$ can, but it is kinked: zero gradient on the branch it drops, Deck 2.3’s dying ReLU in the loss.
- ▶ **Fischer–Burmeister**, $\varphi = \xi_1 + \xi_2 - \sqrt{\xi_1^2 + \xi_2^2}$, is zero exactly on the corner, and not differentiable there. **Session 4 builds it and runs it.**

From Residual to Loss: Zero Is the Label



- ▶ Squared error against a label of zero: $\ell_i = (r_\theta(k_i, z_i) - 0)^2$. The training loop is Deck 2.4's five lines.
- ▶ The policy is evaluated twice, at k and at k' , so gradients flow through both.
- ▶ **No benchmark policy ever enters the loss. Benchmarks are kept for testing.**

The Weighted-Residual Reading

- ▶ Classical **projection methods** write the policy as $\sum_j a_j \phi_j(k)$ with fixed basis functions (Chebyshev polynomials) and choose a to make the residual small.
- ▶ They differ in what “small” means: *collocation* sets $r = 0$ at chosen nodes; *least squares* minimises $\int r^2$; *Galerkin* makes r orthogonal to the basis.
- ▶ The network method is the least-squares member of this family, with two changes: the basis is learned, and the integral is a Monte Carlo average over sampled states.
- ▶ The sampling distribution is the weight function. Where you sample is where you ask for accuracy.
- ▶ **This is Judd’s projection method with a neural basis. The economics has not changed.**

Source: K. Judd, “Projection methods for solving aggregate growth models”, *JET* 58(2), 1992; *Numerical Methods in Economics*, 1998, ch. 11.

What Is Different from Regression on Labels

	Regression on labels	Regression on an equation
data	fixed, finite, noisy	unlimited, drawn fresh, noiseless
overfitting	to the sample's noise	none to noise; errors hide where no state was drawn
the model appears	once, $f_{\theta}(x_i)$	twice, $\pi_{\theta}(k)$ and $\pi_{\theta}(\pi_{\theta}(k))$
a zero loss means	a perfect fit	the first-order condition holds on the sample

- ▶ Against collocation: no grid, no fixed basis, and no linear system. Cost grows with the sample, not with N^d .
- ▶ **Held-out testing survives the change. Its job moves from noise to coverage.**

Three Rungs

Rung	Model	What it tests	Benchmark
0	log utility, $\delta = 1, \beta = 0.96$	the code	$k' = \alpha\beta Ak^\alpha$
1	CRRA $\gamma = 2, \delta = 0.025, \beta = 0.99$	a share that varies	time iteration
2	rung 0 with AR(1) productivity z	the expectation	$k' = \alpha\beta zAk^\alpha$

- ▶ All rungs: $\alpha = 0.33, A = 1$. **CRRA**: $u(c) = c^{1-\gamma}/(1-\gamma)$. Rung 2: $\log z' = 0.9 \log z + 0.05 \varepsilon'$.
- ▶ **Steady state** k^* , where $k' = k$: 0.1798 on rungs 0 and 2, 28.35 on rung 1.
- ▶ **One code path, one new difficulty per rung, and an answer we never trained on.**

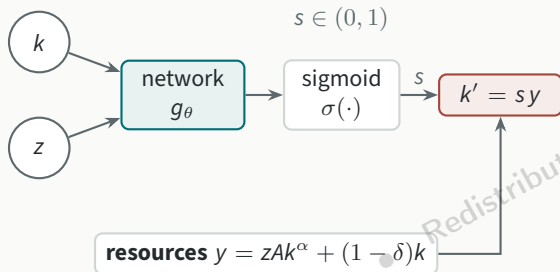
Session 2.5

Feasibility and Expectations

What to build into the network so it cannot cheat, and how to compute $E[\cdot]$



Feasibility by Construction: the Sigmoid Share



- **Feasibility:** $c > 0$ and $k' > 0$. One infeasible state makes $\log c$ undefined and ends training.
- The network outputs a *saving share* $s = \sigma(g_\theta) \in (0, 1)$, so $k' = sy$ and $c = (1 - s)y$ are positive at every θ .
- Inputs are $\log k$ and $\log z$, rescaled to $[-1, 1]$.
- **A constraint written into the architecture cannot be violated, and costs no penalty term.**

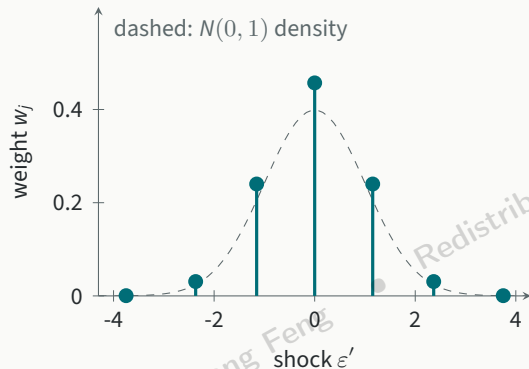
The Stochastic Case: an Expectation Inside the Residual

- ▶ Rung 2: productivity follows $\log z' = \rho \log z + \sigma \varepsilon'$, $\varepsilon' \sim N(0, 1)$, with $\rho = 0.9$, $\sigma = 0.05$. The state is (k, z) .
- ▶ The residual now contains an integral over tomorrow's shock:

$$r_{\theta}(k, z) = \log \frac{\beta E_{\varepsilon'} [u'(c'(\varepsilon')) R'(\varepsilon')]}{u'(c)}.$$

- ▶ Two expectations are in play. The *outer* one, over states, is the loss, and sampling it is harmless.
- ▶ The *inner* one, over ε' , sits inside a logarithm and a square. Sampling it is not harmless.
- ▶ **How the inner expectation is computed decides whether the loss is the right loss.**

Gauss-Hermite, and Why the Loss Stays Differentiable



- **Gauss-Hermite quadrature:**

$E[h(\varepsilon)] \approx \sum_{j=1}^J w_j h(\varepsilon_j)$, with nodes and weights fixed in advance. Exact for polynomials up to degree $2J - 1$.

- $J = 7$ here: seven evaluations of the policy per state, stacked in one batch.
- A fixed weighted sum is an ordinary differentiable function of θ : autograd passes straight through.
- **No sampling noise enters the inner expectation.**

Source: Judd, *Numerical Methods in Economics*, 1998, §7.2. Nodes and weights: Session 2 lab notebook, block D.

Why the Square of a Monte Carlo Mean Is Biased

Inner expectation by	average of (estimate) ²	Var(X)/ n
Monte Carlo, $n = 1$ draw	2.5×10^{-3}	2.5×10^{-3}
Monte Carlo, $n = 10$ draws	2.5×10^{-4}	2.5×10^{-4}
Monte Carlo, $n = 100$ draws	2.5×10^{-5}	2.5×10^{-5}
Gauss–Hermite, 7 nodes	4×10^{-16}	none

- ▶ For a mean $\bar{X}$ of n draws, $E[\bar{X}^2] = (EX)^2 + \text{Var}(X)/n$. The test function here has $EX = 0$.
- ▶ A squared residual with a sampled inner mean is minimised at the wrong policy.
- ▶ **Quadrature removes the bias; the alternative is two independent draws multiplied together.**

Source: Session 2 lab notebook, block D; test function $e^{0.05\varepsilon} - e^{0.05^2/2}$. Maliar, Maliar & Winant, *JME* 122, 2021.

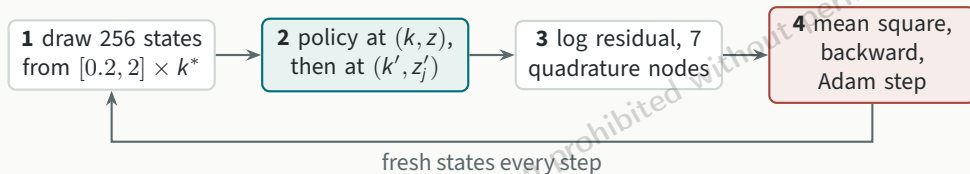
Session 2.5

Algorithm and Validation

Train on residuals; believe only what was tested elsewhere



The Algorithm



- Network: two tanh hidden layers of 32 units (1,153 parameters with one input). Learning rate on a cosine decay.
- Steps: 500 on rung 0, 3,000 on rung 1, 1,500 on rung 2. Seconds on one CPU core: 1.4, 4.6, 4.0.
- **Then stop training and start testing: held-out states, off-domain states, benchmark, steady state.**

Source: Session 2 lab notebook, block D. Azinovic, Gaegauf & Scheidegger, “Deep equilibrium nets”, *IER* 63(4), 2022.

The Code: Policy and Residual

```
def saving_share(self, k, z):
    x = (torch.log(k) - self.log_center) / self.log_scale
    if self.rung.stochastic:
        x = torch.cat([x, torch.log(z) / self.rung.log_z_bound], dim=1)
    return torch.sigmoid(self.net(x))

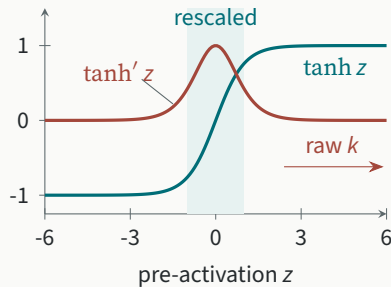
def forward(self, k, z):
    return self.resources(k, z) * self.saving_share(k, z)

def log_residual(model, k, z):
    c, rhs = expected_rhs(model, k, z)
    return torch.log(rhs) + model.rung.gamma * torch.log(c)

loss = log_residual(model, k, z).square().mean()
```

- `expected_rhs` returns c and $\beta E[u'(c')R']$. **Feasibility is line 8; the economics is line 12.**

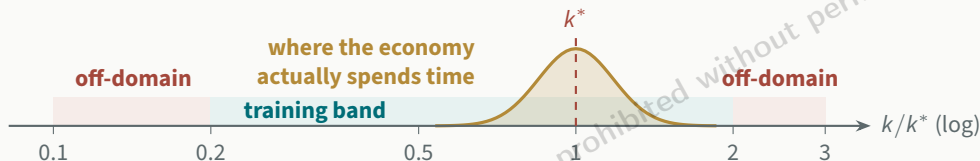
Why the Inputs Are Rescaled



- ▶ Rung 1 runs over $k \in [5, 57]$. A one-input layer starts with weights near $U(-1, 1)$, so raw k makes the pre-activations tens.
- ▶ $\tanh'(10) = 8 \times 10^{-9}$: the first layer saturates and nothing trains — Deck 2.2's vanishing gradient, from input scale, not depth.
- ▶ Mixed scales also stretch the loss contours (Deck 2.3), and Xavier and He assume unit-variance inputs.
- ▶ **The sigmoid share is the same idea at the other end: a bounded, order-one target.**

Source: Glorot & Bengio, *AISTATS* 9, 2010; He et al., *ICCV*, 2015. Session 2 lab notebook, block D.

Where to Sample



- ▶ Training states are uniform in $\log k$ on $[0.2, 2] \times k^*$, and within three standard deviations of $\log z$. Rung 2 itself keeps k inside $[0.72, 1.38] \times k^*$ 95% of the time.
- ▶ **Held-out** states: fresh draws from the same band, from a generator training never used; they test coverage. **Off-domain** states: $[0.1, 0.2) \cup (2, 3] \times k^*$, never asked of the network; they test extrapolation.
- ▶ **The sampling band is a modelling choice: the solution is only claimed where it was trained and tested.**

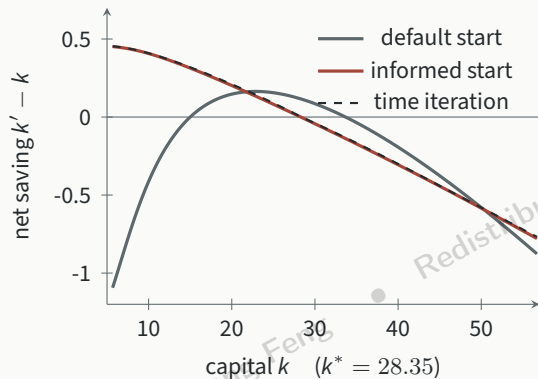
Source: Ergodic density: the stationary distribution of $\log k$ under $k' = \alpha\beta z A k^\alpha$, standard deviation 0.165.

Why Sample at All

- ▶ A grid costs N^d points; drawing n states costs n whatever d is. Sampling is how the curse of dimensionality is dodged operationally, and it is the whole case for the method.
- ▶ Sampling also fixes what “small residual” means. The loss $\min_{\theta} E_{\mu}[r_{\theta}^2]$ is a weighted L^2 norm, and the sampling density μ is the weight.
- ▶ Uniform μ asks for uniform accuracy across the band; the **ergodic distribution** asks for accuracy where the economy spends its time. Different problems, different answers.
- ▶ Neither is right in general. The choice says which states the solution is for, so state it, and report the error apart where you did not sample.
- ▶ **Forward: once the policy generates its own training states, μ moves with θ and stops being a free choice — Sessions 3 and 4.**

Source: K. Judd, *Numerical Methods in Economics*, 1998, ch. 11; Judd, Maliar & Maliar, *Quantitative Economics* 2(2), 2011.

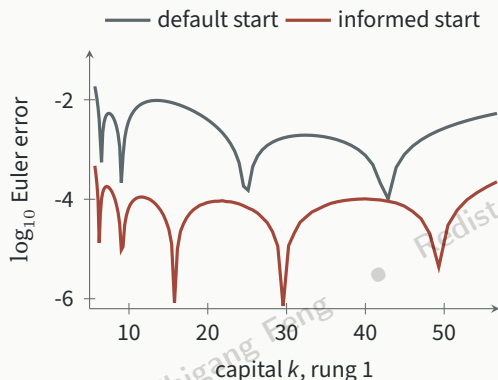
Rung 1, Twice: a Small Residual with the Wrong Policy



- ▶ Default start: $\log_{10}$ max Euler error -1.74 , policy wrong by 25%, and simulated capital falls to 0.06, not 28.35.
- ▶ Informed start (share set to $k^*/y(k^*)$ at initialisation): policy error 0.04%, simulated $k = 28.30$.
- ▶ With $\beta R \approx 1$ each quarter, the residual is nearly flat in the policy.
- ▶ **The loss did not catch this. The benchmark and the steady state did.**

Source: Session 2 lab notebook, block D (rung 1: CRRA $\gamma = 2$, $\delta = 0.025$, $\alpha = 0.33$, $\beta = 0.99$; same seed, same 3,000 steps).

Held-Out and Off-Domain Residuals, on Judd's Scale

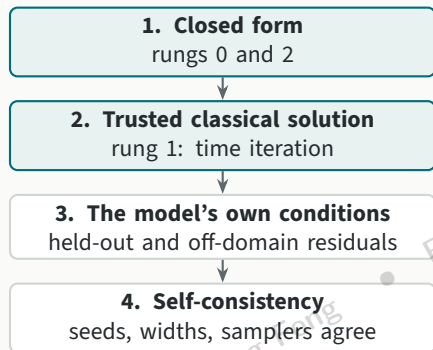


$\log_{10}$ max error	rung 0	rung 1	rung 2
held-out	-1.95	-3.34	-2.15
off-domain	-1.29	-1.59	-1.42

- **Judd's scale:** $\log_{10}$ of the consumption error implied by the residual. -3 is one dollar in a thousand.
- Off-domain, accuracy falls by half a decade to almost two.
- **Report the maximum, and report it where the network was not trained.**

Source: Session 2 lab notebook, block D; calibrations as on "Three Rungs". Judd, *JET* 58(2), 1992.

The Benchmark Hierarchy



- ▶ Validate against the strongest benchmark available, and say which one you had.
- ▶ **Time iteration**: guess tomorrow's policy, solve the Euler equation for today's k' at each grid point, repeat until the policy stops moving. It iterates on the policy itself.
- ▶ **Climb down, never up: prove the code where level 1 exists, then carry the same code further.**

Source: Miao & Feng, *AI for Economists*, ch. 10; W. Coleman, "Equilibrium in a production economy with an income tax", *Econometrica* 59(4), 1991.

The Steady-State Check, and the Gates

Gate	rung 0	rung 1	rung 2
feasible at every tested state	yes	yes	yes
max policy error, held-out (gate: below 1%)	0.64%	0.04%	0.56%
max policy error, off-domain (reported)	3.4%	1.3%	2.9%
simulated against analytical k^* (gate: 1%)	0.47%	0.18%	0.00%

- ▶ Steady-state check: from $0.5 k^*$, apply the learned policy 3,000 times with $z = 1$. A *global* test: local errors that compound show up here.
- ▶ **A gate is fixed before the run. All rungs pass; rung 1 from the default start fails two of three.**

Source: Session 2 lab notebook, block D; calibrations as on “Three Rungs”; policy errors against the closed form (rungs 0, 2) and time iteration (rung 1).

Time Iteration and the Network, Side by Side

Rung 1	time iteration	network
numbers stored	400 grid values	1,153 parameters
seconds, one CPU core	1.5 (292 iterations)	4.6 (3,000 steps)
$\log_{10}$ max Euler error, held-out band	-3.44	-3.34

- ▶ In one dimension the classical method ties on accuracy and wins on the rest. A five-method table from the instructor's quantitative-macro course (another calibration) agrees.
- ▶ A grid costs N^d ; the network's cost grows with the sample. Its case is dimension.
- ▶ **Learn the method where it loses, so that you can trust it where nothing else runs.**

Source: Session 2 lab notebook, block D (rung 1: CRRA $\gamma = 2$, $\delta = 0.025$, $\alpha = 0.33$, $\beta = 0.99$).

What This Deck Did Not Do — and Where Each Idea Goes Next

Built in Session 2

Used next

held-out discipline

every later session

an equation's residual as the loss

Session 4: heterogeneous-agent equilibrium conditions

the network as a *policy*

Session 3: the network as a *value*, and learning by acting

a frozen, dated yield baseline ●

Session 5: does a text signal improve on it?

gates fixed before the run

Session 6: verifying work delegated to agents

- ▶ No value function was approximated today: network and benchmark are both policies.
- ▶ **Next, Deck 2.6: the policy was a plain dense network. What if the architecture itself carried the economics?**